МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра вычислительной техники

ОТЧЕТ

по лабораторной работе 1
по дисциплине «Операционные системы»
по теме «Управление файловой системой»

Студент гр. 4311		Левицкий М.Ю.
Преподаватель		Тимофеев А.В.

Санкт-Петербург
2026

Цель работы.

Исследовать управление файловой системой с помощьюWin32API и AIO Linux.

Задание 1.1.

Используемые функции Win32 API

1. GetLogicalDrives

Функция предназначена для получения списка доступных логических дисков в системе.

Она возвращает 32-битное число, где каждый бит соответствует определённому диску (от A до Z).

- Если бит установлен в 1 — диск существует
- Если 0 — диск отсутствует

В программе эта функция используется для формирования списка всех доступных дисков путём проверки каждого бита и сопоставления его с буквой диска.

2. GetDriveTypeA

Функция определяет тип указанного диска.

Принимает путь к диску (например, "C:\\") и возвращает значение, соответствующее типу:

- DRIVE_FIXED — жёсткий диск
- DRIVE_REMOVABLE — съёмный носитель (флешка)
- DRIVE_CDROM — CD/DVD
- DRIVE_REMOTE — сетевой диск
- DRIVE_UNKNOWN — тип неизвестен

В программе используется для вывода пользователю информации о типе выбранного диска.

3. GetVolumeInformationA

Функция предоставляет подробную информацию о файловом томе.

Позволяет получить:

- имя тома (метку)
- файловую систему (например, NTFS, FAT32)
- серийный номер тома
- максимальную длину имени файла
- флаги файловой системы

В программе используется для отображения пользователю метки тома и типа файловой системы.

4. GetDiskFreeSpaceA

Функция возвращает информацию о дисковом пространстве.

Определяет:

- количество секторов в кластере
- размер сектора в байтах
- число свободных кластеров
- общее количество кластеров

На основе этих данных в программе вычисляется объём свободного места на диске по формуле:

Свободное место = freeClusters × sectorsPerCluster × bytesPerSector

5. CreateDirectoryA

Функция создаёт новый каталог по указанному пути.

- Если каталог успешно создан — возвращает ненулевое значение
- В случае ошибки — возвращает 0

В программе используется для создания новой директории, введённой пользователем.

6. RemoveDirectoryA

Удаляет указанный каталог.

Важно:

- каталог должен быть пустым
- иначе функция завершится с ошибкой

В программе применяется для удаления пользовательского каталога.

7. CreateFileA

Одна из универсальных функций Win32 API для работы с файлами.

Используется для:

- создания файла
- открытия существующего файла
- чтения/записи

В данной работе применяется с параметром CREATE_NEW, что означает:

- создать новый файл
- если файл уже существует — возникнет ошибка

Функция возвращает дескриптор файла (HANDLE), который используется в дальнейшей работе.

8. CloseHandle

Функция закрывает ранее открытый дескриптор (например, файл).

Это необходимо:

- для освобождения системных ресурсов
- для корректного завершения работы с файлом

В программе вызывается после создания файла.

9. CopyFileA

Функция копирует файл из одного пути в другой.

Параметры:

- исходный файл
- конечный файл
- флаг перезаписи

В программе используется режим, при котором копирование запрещено, если файл назначения уже существует.

10. MoveFileExA

Функция перемещает файл или изменяет его имя.

Флаг MOVEFILE_COPY_ALLOWED позволяет:

- копировать файл, если перемещение между разными дисками невозможно

Используется для перемещения файлов по указанному пользователем пути.

11. GetFileAttributesA

Функция возвращает атрибуты файла или каталога.

Возможные атрибуты:

- FILE_ATTRIBUTE_READONLY — только для чтения
- FILE_ATTRIBUTE_HIDDEN — скрытый
- FILE_ATTRIBUTE_SYSTEM — системный
- FILE_ATTRIBUTE_ARCHIVE — архивный

В программе производится побитовая проверка атрибутов и их вывод пользователю.

12. SetFileAttributesA

Функция изменяет атрибуты файла.

В программе используется для установки атрибута:

- FILE_ATTRIBUTE_READONLY (только для чтения)

Для этого:

- сначала считываются текущие атрибуты
- затем добавляется новый с помощью побитовой операции OR

Скриншоты работы программы.

Рисунок 1 – Меню

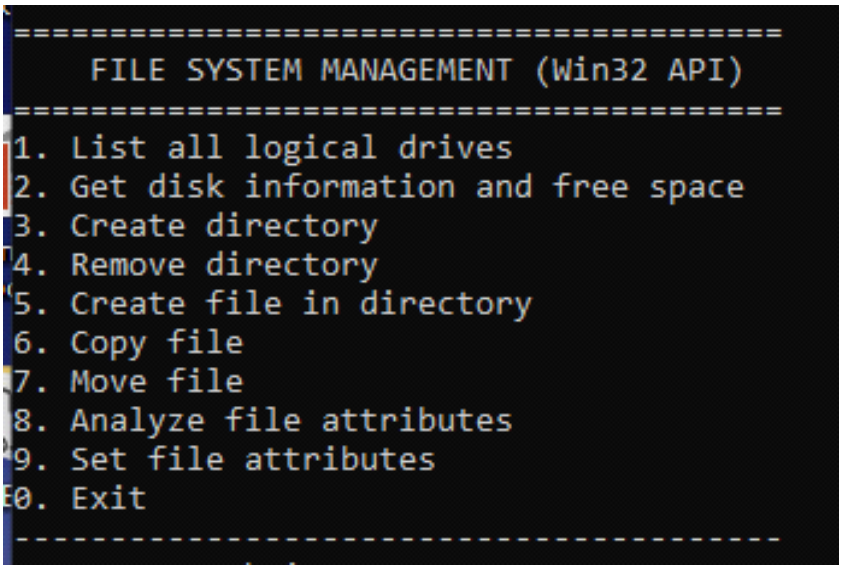


Рисунок 2 – Вывод доступных дисков

```
Enter your choice: 1

=== List Drives ===
GetLogicalDrives: C: D:
GetLogicalDriveStrings: C:\ D:\
```

Рисунок 3 – Вывод информации о выбранном диске

```
Enter your choice: 2
Enter drive path (e.g., C:\): C:\

=== Disk Info ===
Drive type: 3
Volume: (no label)
File system: NTFS
Serial number: de0cc140
Max component length: 255
File system flags: 65472255
Sectors per cluster: 8
Bytes per sector: 512
Cluster size: 4096 bytes
Total clusters: 244104190
Free clusters: 148766731
Total space: 931 GB
Free space: 567 GB
```

Рисунок 4 – Создание директории

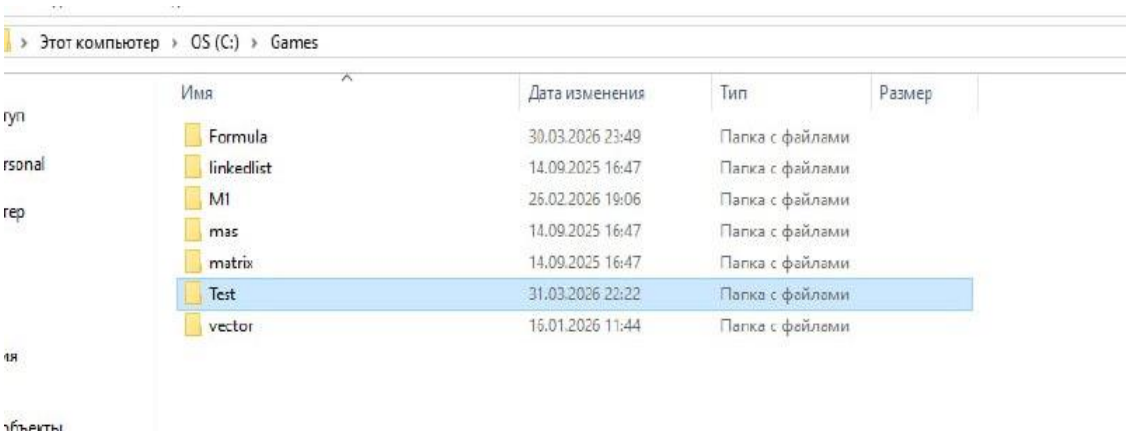


Рисунок 5 – Удаление директории

```
=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Удалить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 4
Введите путь каталога: C:\Games\Test
Каталог удален.
```

Рисунок 6 – Создание файла в выбранной директории

```
=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Удалить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 3
Input path of new directory: C:\Games\Test
Каталог создан.

=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Удалить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 5
Введите путь нового файла: C:\Games\Test\File.txt
Файл создан.

=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Удалить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор:
```

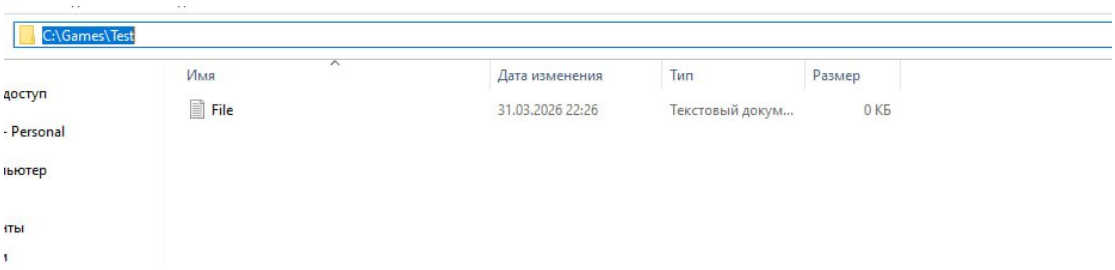
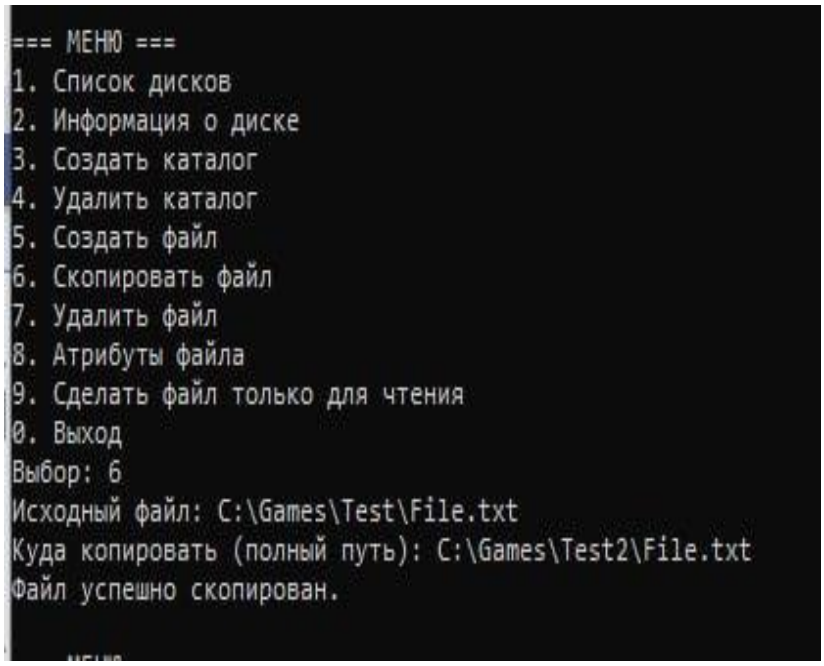


Рисунок 7 – Копирование файла из одной директории в другую



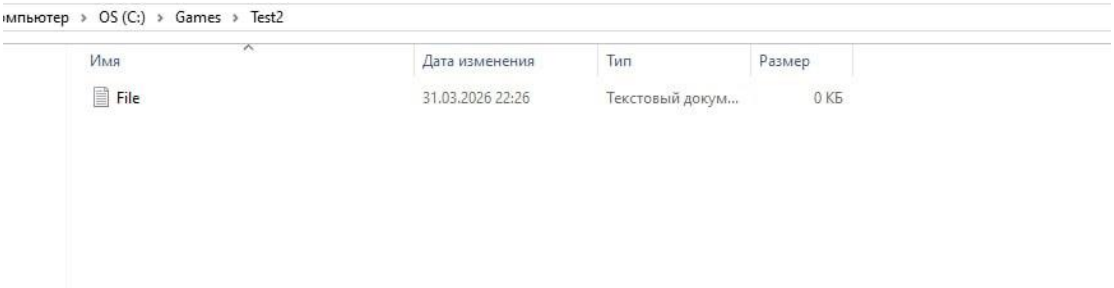
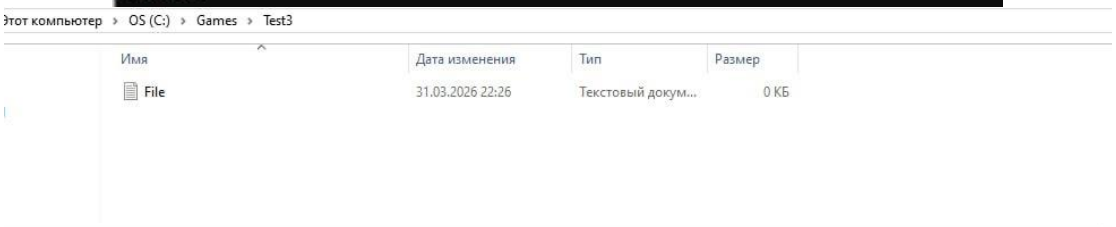


Рисунок 8 – Перемещение файла из одной директории в другую

```
=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Удалить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 3
Input path of new directory: C:\Games\Test3
Каталог создан.

=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Удалить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 7
Исходный файл: C:\Games\Test\File.txt
Куда переместить (полный путь): C:\Games\Test3\File.txt
Файл перемещен.

=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Удалить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор:
```



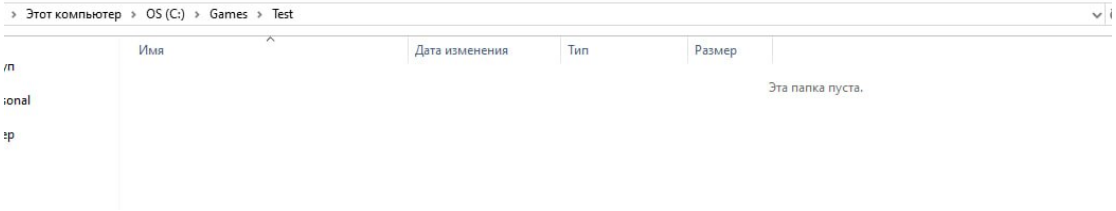


Рисунок 9 – Вывод атрибутов выбранного файла

```
=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Переместить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 8
Введите путь файла: C:\Games\Test2\File.txt
Атрибуты:
READONLY
ARCHIVE

=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Переместить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 9
Введите путь файла: C:\Games\Test2\File.txt
1 - установить READONLY
2 - убрать READONLY
2
Атрибут изменён.

=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Переместить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 8
Введите путь файла: C:\Games\Test2\File.txt
Атрибуты:
ARCHIVE

=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
```

Рисунок 10 – Смена атрибутов файла и результат корректной смены

```
=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Переместить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 8
Введите путь файла: C:\Games\Test2\File.txt
Атрибуты:
READONLY
ARCHIVE

=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Переместить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 9
Введите путь файла: C:\Games\Test2\File.txt
1 - установить READONLY
2 - убрать READONLY
2
Атрибут изменён.

=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
4. Удалить каталог
5. Создать файл
6. Скопировать файл
7. Переместить файл
8. Атрибуты файла
9. Сделать файл только для чтения
0. Выход
Выбор: 8
Введите путь файла: C:\Games\Test2\File.txt
Атрибуты:
ARCHIVE

=== МЕНЮ ===
1. Список дисков
2. Информация о диске
3. Создать каталог
```

Рисунок 11 – Выход из меню

```
Enter your choice: 0
Exiting program...
```

Задание 1.2.

Копирование файла с помощью операций перекрывающегося ввода-вывода.

Функции AIO Linux, используемые для выполнения задания.

- Структура aiocb – блок управления асинхронным вводом-выводом. Задаёт параметры, которые управляют операцией ввода-вывода: aio_fildes – файловый дескриптор из которого будут читаться данные (если aio_read) или в который будут записываться данные (если aio_write), aio_buf – буфер, используемый для пересылки данных, aio_offset – файловое смещение (место в файле, с которого будет выполняться операция ввода-вывода для блока данных), aio_nbytes – размер буфера aio_buf.

- aio_read – ставить запрос на чтение в очередь.

- aio_write – ставит запрос на запись в очередь.

- aio_error – возвращает информацию о состоянии поставленного в очередь запроса ввода-вывода.

- aio_return – возвращает информацию о выполненном запросе ввода-вывода (сколько байт прочитано/записано).

- aio_suspend – приостанавливает программу до того момента, пока не выполнится хотя-бы 1 запрос на ввод-вывод.

- fstat – возвращает информацию о состоянии открытого файла. Позволяет получить размер файла.

Код написанной программы с использованием данных функций представлен в Приложении Б отчёта.

О программе.

Данная программа имеет 2 режима работы:

- Асинхронные чтение и запись для фиксированного блока 4096 байт с подробным выводом в консоль о начале/окончании чтения, о начале/окончании записи, о месте с которого начал читаться/записываться блок и размере блока. Время, вычисляемое в данном режиме работы, будет отличаться от реального времени, затраченного на асинхронные операции, так как программа, кроме самого чтения и записи, тратит ещё некоторое время на вывод информации в консоль. Этот режим подойдёт для ознакомления с принципом работы асинхронных операций.

- Асинхронные чтения и запись для различных блоков от 4096 до 2097152 байт. Данная программа не выводит в консоль процесс своей работы, но показывает реальное время, затраченное на асинхронные операции. Этот режим подойдёт для проведения различных тестов и анализа полученных результатов.

Зависимость скорости копирования от размера блока при 1 асинхронной операции ввода-вывода.

В данном эксперименте были использованы следующие блоки данных (байт): 4096, 8192, 16384, 32768, 65536, 131072, 262144, 524288, 1048576, 2097152.

Полученные результаты представлены на скриншотах ниже.

```
========================================
         Async File Copy Program
========================================

Enter source file path: test_source.txt
Enter destination file path: output.txt
Enter number of parallel operations (1-16): 1
--- Enter type of work ---
1. Test only one block size with detailed read and write output
2. Test more block size with no detailed output
2

===> Configuration <===
Source: test_source.txt
Destination: output.txt
Source file size: 1073741824 bytes (1024.00 MB)
========================


===> Copy Results №1<===
------------------------
Block size: 4096 bytes
Parallel operations: 1
------------------------
Bytes copied: 1073741824
Time elapsed: 6.54 seconds
Speed: 156.60 MB/s
========================

===> File Comparison <===
Files are identical!
=========================


===> Copy Results №2<===
------------------------
Block size: 8192 bytes
Parallel operations: 1
------------------------
Bytes copied: 1073741824
Time elapsed: 3.14 seconds
Speed: 326.43 MB/s
========================

===> File Comparison <===
Files are identical!
=========================
```

Рисунок 12 – Результат для тестов 1 и 2

```
===> Copy Results №3<===
-------------------------
Block size: 16384 bytes
Parallel operations: 1
-------------------------
Bytes copied: 1073741824
Time elapsed: 1.83 seconds
Speed: 558.65 MB/s
=========================

===> File Comparison <===
Files are identical!
==========================


===> Copy Results №4<===
-------------------------
Block size: 32768 bytes
Parallel operations: 1
-------------------------
Bytes copied: 1073741824
Time elapsed: 1.14 seconds
Speed: 899.82 MB/s
=========================

===> File Comparison <===
Files are identical!
==========================


===> Copy Results №5<===
-------------------------
Block size: 65536 bytes
Parallel operations: 1
-------------------------
Bytes copied: 1073741824
Time elapsed: 0.82 seconds
Speed: 1250.31 MB/s
=========================

===> File Comparison <===
Files are identical!
==========================
```

```
===> Copy Results №6<===
------------------------
Block size: 131072 bytes
Parallel operations: 1
------------------------
Bytes copied: 1073741824
Time elapsed: 0.58 seconds
Speed: 1765.52 MB/s
========================

===> File Comparison <===
Files are identical!
=========================


===> Copy Results №7<===
------------------------
Block size: 262144 bytes
Parallel operations: 1
------------------------
Bytes copied: 1073741824
Time elapsed: 0.51 seconds
Speed: 2003.91 MB/s
========================

===> File Comparison <===
Files are identical!
=========================


===> Copy Results №8<===
------------------------
Block size: 524288 bytes
Parallel operations: 1
------------------------
Bytes copied: 1073741824
Time elapsed: 0.43 seconds
Speed: 2392.52 MB/s
========================

===> File Comparison <===
Files are identical!
=========================
```

Рисунок 13 – Результат для тестов 3-8

```
===> Copy Results №9<===
-------------------------
Block size: 1048576 bytes
Parallel operations: 1
-------------------------
Bytes copied: 1073741824
Time elapsed: 1.07 seconds
Speed: 959.70 MB/s
========================

===> File Comparison <===
Files are identical!
=========================


===> Copy Results №10<===
-------------------------
Block size: 2097152 bytes
Parallel operations: 1
-------------------------
Bytes copied: 1073741824
Time elapsed: 0.77 seconds
Speed: 1326.42 MB/s
========================

===> File Comparison <===
Files are identical!
=========================
```

Рисунок 14 – Результат для тестов 9 и 10

Ось X – размер блока данных (байт) Ось Y – время копирования (МБ/с)

Зависимость скорости копирования от количества перекрывающихся операций ввода-вывода.

Данный эксперимент проводился с использованием блока 16384 байта и текстовым файлом в 1 ГБ.

Результаты проведённых тестов представлены на скриншотах ниже.

```
===> Configuration <===
Source: test_source.txt
Destination: output.txt
Source file size: 1073741824 bytes (1024.00 MB)
========================


===> Copy Results №3 <===
--------------------------
Block size: 16384 bytes
Parallel operations: 1
--------------------------
Bytes copied: 1073741824
Time elapsed: 2.38 seconds
Speed: 429.71 MB/s
========================

===> File Comparison <===
Files are identical!
==========================
```

Рисунок 15 – 1 асинхронная операция

```
===> Configuration <===
Source: test_source.txt
Destination: output.txt
Source file size: 1073741824 bytes (1024.00 MB)
========================


===> Copy Results №3 <===
-------------------------
Block size: 16384 bytes
Parallel operations: 2
-------------------------
Bytes copied: 1073741824
Time elapsed: 2.50 seconds
Speed: 409.27 MB/s
========================

===> File Comparison <===
Files are identical!
==========================
```

Рисунок 16 – 2 асинхронная операция

```
===> Configuration <===
Source: test_source.txt
Destination: output.txt
Source file size: 1073741824 bytes (1024.00 MB)
========================


===> Copy Results №3 <===
-------------------------
Block size: 16384 bytes
Parallel operations: 4
-------------------------
Bytes copied: 1073741824
Time elapsed: 1.16 seconds
Speed: 886.58 MB/s
========================

===> File Comparison <===
Files are identical!
==========================
```

Рисунок 17 – 4 асинхронных операции

```
===> Configuration <===
Source: test_source.txt
Destination: output.txt
Source file size: 1073741824 bytes (1024.00 MB)
========================


===> Copy Results №3 <===
--------------------------
Block size: 16384 bytes
Parallel operations: 8
--------------------------
Bytes copied: 1073741824
Time elapsed: 1.17 seconds
Speed: 875.21 MB/s
========================

===> File Comparison <===
Files are identical!
==========================
```

Рисунок 18 – 8 асинхронных операций

```
===> Configuration <===
Source: test_source.txt
Destination: output.txt
Source file size: 1073741824 bytes (1024.00 MB)
========================


===> Copy Results №3 <===
--------------------------
Block size: 16384 bytes
Parallel operations: 12
--------------------------
Bytes copied: 1073741824
Time elapsed: 1.31 seconds
Speed: 778.71 MB/s
========================

===> File Comparison <===
Files are identical!
==========================
```

Рисунок 19 – 12 асинхронных операций

```
===> Configuration <===
Source: test_source.txt
Destination: output.txt
Source file size: 1073741824 bytes (1024.00 MB)
========================


===> Copy Results №3 <===
--------------------------
Block size: 16384 bytes
Parallel operations: 16
--------------------------
Bytes copied: 1073741824
Time elapsed: 1.33 seconds
Speed: 768.19 MB/s
==========================

===> File Comparison <===
Files are identical!
==========================
```

Рисунок 20 – 16 асинхронных операций

Ось X – количество операций Ось Y – время копирования (МБ/с)

Вывод.

В первом задании были успешно освоены основные функции Win32 API для управления файловой системой. Разработанная программа позволяет получать информацию о дисках, их типе, файловой системе, а также

выполнять операции с каталогами и файлами: создание, удаление, копирование, перемещение, изменение атрибутов и временных меток.

Во втором задании исследовался асинхронный ввод-вывод (AIO) в Linux. Экспериментально установлена зависимость времени копирования файла от размера блока данных. По результатам проведённых экспериментов можно установить, что при увеличении размера блока скорость копирования возрастает, однако при достижении определённой отметки (в данном эксперименте — 1048576 байт) скорость копирования немного падает и после этого перестаёт расти так же интенсивно, как на предыдущих этапах. Схожая тенденция наблюдается и в эксперименте с зависимостью скорости копирования от числа перекрывающихся операций: при достижении определённого количества операций (в эксперименте — 4 операции) скорость копирования снижается и перестаёт стремительно возрастать. Отсюда можно сделать вывод, что существует некоторое оптимальное соотношение между размером блока и числом операций, при котором достигается наибольшая скорость копирования.

ПРИЛОЖЕНИЕ А

Программа для задания 1.1.

```
#include <windows.h>
```

```
#include <iostream>
#include <string>

using namespace std;

void listDrives() {
    DWORD drives = GetLogicalDrives();
    cout << "Доступные диски:\n";

    for (int i = 0; i < 26; i++) {
        if (drives & (1 << i)) {
            char letter = 'A' + i;
            cout << letter << ":\\\n";
        }
    }
}

void driveInfo() {
    string disk;
    cout << "Введите диск (например C:\\): ";
    cin >> disk;

    UINT type = GetDriveTypeA(disk.c_str());

    cout << "Тип диска: ";
    switch (type) {
    case DRIVE_FIXED: cout << "Жесткий диск\n"; break;
    case DRIVE_REMOVABLE: cout << "Съёмный диск\n"; break;
    case DRIVE_CDROM: cout << "CD/DVD\n"; break;
    default: cout << "Другой\n";
```

```
}

char volumeName[MAX_PATH];
char fileSystem[MAX_PATH];
DWORD serial, maxCompLen, flags;

if (GetVolumeInformationA(
    disk.c_str(),
    volumeName, MAX_PATH,
    &serial,
    &maxCompLen,
    &flags,
    fileSystem, MAX_PATH)) {

    cout << "Метка тома: " << volumeName << endl;
    cout << "Файловая система: " << fileSystem << endl;
}

DWORD sectorsPerCluster, bytesPerSector, freeClusters, totalClusters;

if (GetDiskFreeSpaceA(
    disk.c_str(),
    &sectorsPerCluster,
    &bytesPerSector,
    &freeClusters,
    &totalClusters)) {

    ULONGLONG freeSpace =
        (ULONGLONG)freeClusters * sectorsPerCluster * bytesPerSector;
```

```
        cout << "Free space (байт): " << freeSpace << endl;
    }
}

void createDir() {
    string path;
    cout << "Input path of new directory: ";
    cin >> path;

    if (CreateDirectoryA(path.c_str(), NULL))
        cout << "Каталог создан.\n";
    else
        cout << "Ошибка создания.\n";
}

void removeDir() {
    string path;
    cout << "Введите путь каталога: ";
    cin >> path;

    if (RemoveDirectoryA(path.c_str()))
        cout << "Каталог удален.\n";
    else
        cout << "Ошибка удаления.\n";
}

void createFileFunc() {
    string path;
    cout << "Введите путь нового файла: ";
    cin >> path;
```

```
    HANDLE hFile = CreateFileA(
        path.c_str(),
        GENERIC_WRITE,
        0,
        NULL,
        CREATE_NEW,
        FILE_ATTRIBUTE_NORMAL,
        NULL);

    if (hFile != INVALID_HANDLE_VALUE) {
        cout << "Файл создан.\n";
        CloseHandle(hFile);
    }
    else {
        cout << "Ошибка создания файла.\n";
    }
}

void copyFileFunc() {
    string src, dst;
    cout << "Исходный файл: ";
    cin >> src;
    cout << "Куда копировать: ";
    cin >> dst;

    if (CopyFileA(src.c_str(), dst.c_str(), TRUE))
        cout << "Файл скопирован.\n";
    else
        cout << "Ошибка копирования.\n";
```

```
}

void moveFileFunc() {
    string src, dst;
    cout << "Исходный файл: ";
    cin >> src;
    cout << "Куда переместить: ";
    cin >> dst;

    if (MoveFileExA(src.c_str(), dst.c_str(),
MOVEFILE_COPY_ALLOWED))
        cout << "Файл перемещен.\n";
    else
        cout << "Ошибка перемещения.\n";
}

void fileAttributes() {
    string path;
    cout << "Введите путь файла: ";
    cin >> path;

    DWORD attrs = GetFileAttributesA(path.c_str());

    if (attrs == INVALID_FILE_ATTRIBUTES) {
        cout << "Ошибка.\n";
        return;
    }

    cout << "Атрибуты:\n";
    if (attrs & FILE_ATTRIBUTE_READONLY) cout << "READONLY\n";
```

```
        if (attrs & FILE_ATTRIBUTE_HIDDEN) cout << "HIDDEN\n";
        if (attrs & FILE_ATTRIBUTE_SYSTEM) cout << "SYSTEM\n";
        if (attrs & FILE_ATTRIBUTE_ARCHIVE) cout << "ARCHIVE\n";
    }

    void setReadOnly() {
        string path;
        cout << "Введите путь файла: ";
        cin >> path;

        DWORD attrs = GetFileAttributesA(path.c_str());
        if (attrs == INVALID_FILE_ATTRIBUTES) {
            cout << "Ошибка.\n";
            return;
        }

        if (SetFileAttributesA(path.c_str(), attrs |
FILE_ATTRIBUTE_READONLY))
            cout << "Атрибут READONLY установлен.\n";
        else
            cout << "Ошибка изменения атрибутов.\n";
    }

    int main() {
        setlocale(LC_CTYPE, "rus");
        int choice;

        do {
            cout << "\n=== МЕНЮ ===\n";
            cout << "1. Список дисков\n";
```

```
        cout << "2. Информация о диске\n";
        cout << "3. Создать каталог\n";
        cout << "4. Удалить каталог\n";
        cout << "5. Создать файл\n";
        cout << "6. Скопировать файл\n";
        cout << "7. Удалить файл\n";
        cout << "8. Атрибуты файла\n";
        cout << "9. Сделать файл только для чтения\n";
        cout << "0. Выход\n";
        cout << "Выбор: ";
        cin >> choice;

        switch (choice) {
        case 1: listDrives(); break;
        case 2: driveInfo(); break;
        case 3: createDir(); break;
        case 4: removeDir(); break;
        case 5: createFileFunc(); break;
        case 6: copyFileFunc(); break;
        case 7: moveFileFunc(); break;
        case 8: fileAttributes(); break;
        case 9: setReadOnly(); break;
        }

    } while (choice != 0);

    return 0;
}
```

ПРИЛОЖЕНИЕ Б

Программа для задания 1.2.

```
#include <iostream>
#include <fstream>
#include <cstring>
#include <cerrno>
#include <aio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <string>
#include <iomanip>
#include <chrono>

using namespace std;
using namespace chrono;

struct Operation {
    aiocb request;
    char* buf;
    int is_write;
    Operation* next;
};

off_t g_file_size = 0;
off_t g_copied = 0;
off_t g_offset = 0;

// ---------- open ----------
int open_read_file(const string& name) {
    int fd = open(name.c_str(), O_RDONLY | O_NONBLOCK, 0666);
```

```
    if (fd == -1)
        cerr << "Open read error: " << strerror(errno) << endl;
    return fd;
}

int open_write_file(const string& name) {
    int fd = open(name.c_str(), O_CREAT | O_WRONLY | O_TRUNC | O_NONBLOCK,
0666);
    if (fd == -1)
        cerr << "Open write error: " << strerror(errno) << endl;
    return fd;
}

// ---------- size ----------
off_t file_size_get(int fd) {
    struct stat s;
    if (fstat(fd, &s) == -1) {
        cerr << "fstat error: " << strerror(errno) << endl;
        return -1;
    }
    return s.st_size;
}

// ---------- close ----------
void close_fd(int fd) {
    if (fd != -1) close(fd);
}

// ---------- compare ----------
bool files_equal(const string& f1, const string& f2) {
    ifstream a(f1, ios::binary);
    ifstream b(f2, ios::binary);

    if (!a || !b) return false;

    const size_t SIZE = 8192;
    char x[SIZE], y[SIZE];

    while (true) {
```

```
            a.read(x, SIZE);
            b.read(y, SIZE);

            if (a.gcount() != b.gcount()) return false;
            if (a.gcount() == 0) return true;

            if (memcmp(x, y, a.gcount()) != 0) return false;
        }
    }

    // ---------- copy ----------
    bool async_copy(int fd_src, int fd_dst, size_t block, int max_ops, int
verbose) {

        Operation* head = nullptr;
        int active = 0;

        while (g_copied < g_file_size) {

            // запуск чтения
            while (g_offset < g_file_size && active < max_ops) {

                size_t sz = min(block, (size_t)(g_file_size - g_offset));

                Operation* op = new Operation();
                op->buf = new char[sz];
                op->is_write = 0;

                memset(&op->request, 0, sizeof(aiocb));

                op->request.aio_fildes = fd_src;
                op->request.aio_buf = op->buf;
                op->request.aio_nbytes = sz;
                op->request.aio_offset = g_offset;

                if (aio_read(&op->request) == -1) {
                    cerr << "aio_read error: " << strerror(errno) << endl;
                    delete op;
                    return false;
```

```
        }

        op->next = head;
        head = op;

        active++;
        g_offset += sz;

        if (verbose) {
            cout << "[START] Read offset " << op->request.aio_offset
                 << " size " << op->request.aio_nbytes << endl;
        }
    }

    if (active == 0) break;

    // формируем список
    int count = 0;
    for (Operation* cur = head; cur; cur = cur->next) count++;

    aiocb** list = new aiocb*[count];

    int i = 0;
    for (Operation* cur = head; cur; cur = cur->next)
        list[i++] = &cur->request;

    if (aio_suspend(list, count, nullptr) == -1) {
        cerr << "aio_suspend error: " << strerror(errno) << endl;
        delete[] list;
        return false;
    }

    delete[] list;

    // обработка
    Operation** ptr = &head;

    while (*ptr) {
```

```
    Operation* cur = *ptr;
    int err = aio_error(&cur->request);

    if (err != EINPROGRESS) {

        if (err != 0) {
            cerr << "aio_error: " << strerror(err) << endl;
            return false;
        }

        int bytes = aio_return(&cur->request);
        if (bytes == -1) {
            cerr << "aio_return error: " << strerror(errno) <<
endl;
            return false;
        }

        if (!cur->is_write) {
            // чтение завершено → запись

    if (verbose) {
        cout << "[DONE] Read offset "
            << cur->request.aio_offset
            << " bytes " << bytes << endl;
    }

    cur->request.aio_fildes = fd_dst;
    cur->request.aio_nbytes = bytes;
    cur->is_write = 1;

    if (aio_write(&cur->request) == -1) {
        cerr << "aio_write error: " << strerror(errno) << endl;
        return false;
    }

    if (verbose) {
        cout << "[START] Write offset "
            << cur->request.aio_offset
            << " size " << bytes << endl;
    }
```

```
                ptr = &cur->next;
            }
            else {
                // запись завершена

                if (verbose) {
                    cout << "[DONE] Write offset "
                        << cur->request.aio_offset
                        << " bytes " << bytes << endl;
                }

                g_copied += bytes;
                active--;

                *ptr = cur->next;

                delete[] cur->buf;
                delete cur;
            }
        }
        else {
            ptr = &cur->next;
        }
    }
}

    return true;
}

// ---------- main ----------
int main() {

    string src, dst;
    int ops, mode, count;

    const int BLOCKS[10] = {
        4096,8192,16384,32768,65536,
        131072,262144,524288,1048576,2097152
    };
```

```
cout << "Source file: ";
cin >> src;

cout << "Destination file: ";
cin >> dst;

cout << "Operations (1-16): ";
cin >> ops;

if (ops < 1) ops = 1;
if (ops > 16) ops = 16;

int fd1 = open_read_file(src);
int fd2 = open_write_file(dst);

if (fd1 == -1 || fd2 == -1) return 1;

g_file_size = file_size_get(fd1);
if (g_file_size == -1) return 1;

cout << "1 - detailed\n2 - test\n";
cin >> mode;

if (mode == 1) count = 1;
else if (mode == 2) count = 10;
else {
    cout << "Wrong mode\n";
    return 1;
}

for (int i = 0; i < count; i++) {

    g_copied = 0;
    g_offset = 0;

    cout << "\nBlock size: " << BLOCKS[i] << endl;

    auto t1 = high_resolution_clock::now();
```

```
        bool ok = async_copy(fd1, fd2, BLOCKS[i], ops, mode);

        auto t2 = high_resolution_clock::now();

        double time =
            duration_cast<milliseconds>(t2 - t1).count() / 1000.0;

        cout << "Copied: " << g_copied << endl;
        cout << "Time: " << fixed << setprecision(2) << time << endl;
        cout << "Speed: "
             << (g_copied / (1024.0 * 1024.0)) / time
             << " MB/s\n";

        cout << (files_equal(src, dst) ? "OK" : "FAIL") << endl;
    }

    close_fd(fd1);
    close_fd(fd2);

    return 0;
}
```